

```

});

static bool writeable_reg(struct device *dev, unsigned int
reg)
{
    if(reg >= 0x20 && reg <= 0x4F)
        return true;

    if(reg >= 0x60 && reg <= 0x7F)
        return true;

    return false;
}

static bool readable_reg(struct device *dev, unsigned int
reg)
{
    if(reg >= 0x20 && reg <= 0x4F)
        return true;

    if(reg >= 0x60 && reg <= 0x7F)
        return true;

    return false;
}

static int custom_drv_probe(struct spi_device *dev)
{
    struct regmap_config config;
    struct custom_drv_private_struct *priv;
    unsigned int data;
    //configure the regmap configuration
    memset(&config, 0, sizeof(config));
    config.reg_bits = 8;
    config.val_bits = 8;
    config.write_flag_mask = 0x80;
    config.max_register = 0x80;
    config.fast_io = true;
    config.writeable_reg = drv_writeable_reg;
    config.readable_reg = drv_readable_reg;
    //only set below two things if
    //writeable_reg
    //and readable_reg is not set
    //config.wr_table = drv_wr_table;
    //config.rd_table = drv_rd_table;

    //allocate the private data structures as
    //priv = devm_kzalloc
    //Init the regmap spi configuration
    priv->map = devm_regmap_init_spi(dev,
&config);
    //devm_regmap_init_i2c in case of i2c bus
    //following operation will remain same in
    //case of both i2c and spi or other bus
    //read from the device, data variable will //contain device

```

```

data
regmap_read(priv->map, 0x23, &data);
data = 0x24;
//write to the device
regmap_write(priv->map, 0x23, data);
if(regmap_read(priv->map, 0x85, &data)
< 0)
{
    ///error since address is out of range
}
return 0;
}

```

You can see in the above example how redundant code in the drivers based on different bus sub-systems becomes similar code, which makes driver writing and maintenance easier. Cache support is also available in the current regmap sub-system.

Caching avoids directly performing operations on the device. Instead, it caches the value transferring between the device and driver, and uses it as future reference. Initially, caches used just flat arrays, which were not good for 32-bit addresses. Later, this issue was solved with better cache types:

- *rbtree* stores blocks of contiguous registers in a red/black tree
 - *Compressed* stores blocks of compressed data
- Both rely on the existing kernel library:

```
enum regcache_type cache_type;
```

Support for tracepoint is available in regmap. For more information, see [debugfs/trace/events/regmap](#):

```
regmap_reg_write 0-001b reg=3b val=1a
regmap_reg_read 0-001b reg=1d val=1d
```

You can also define `LOG_DEVICE` for early init log 

References

- [1] DS1343 driver file: <https://git.kernel.org/cgit/linux/kernel/git/torvalds/linux.git/tree/drivers rtc/rtc-ds1343.c>
- [2] http://elinux.org/images/a/a3/Regmap_The_Power_of_Subsystems_and_Abstractions.pdf
- [3] <http://free-electrons.com/pub/conferences/2015/elce/belloni-mfd-regmap-syscon/belloni-mfd-regmap-syscon.pdf>
- [4] <http://www.sirena.org.uk/2011/09/30/regmap-a-register-map-abstraction-for-the-linux-kernel/>

By: Raghavendra Chandra Ganiga

The author is an embedded firmware development engineer at K2 Technosoft Pvt Ltd, Pune. His interests are microcontrollers, networking firmware, RTOS development and Linux device drivers. You can contact him at ravi23ganiga@gmail.com.